Module 16: Manage Automation Execution Environments

4 HOURS | 30% THEORY | 70% HANDS-ON

01

Understand EEs

Purpose of Execution Environments

02

Build

Custom EEs with ansible-builder

03

Test

Locally with ansible-navigator

04

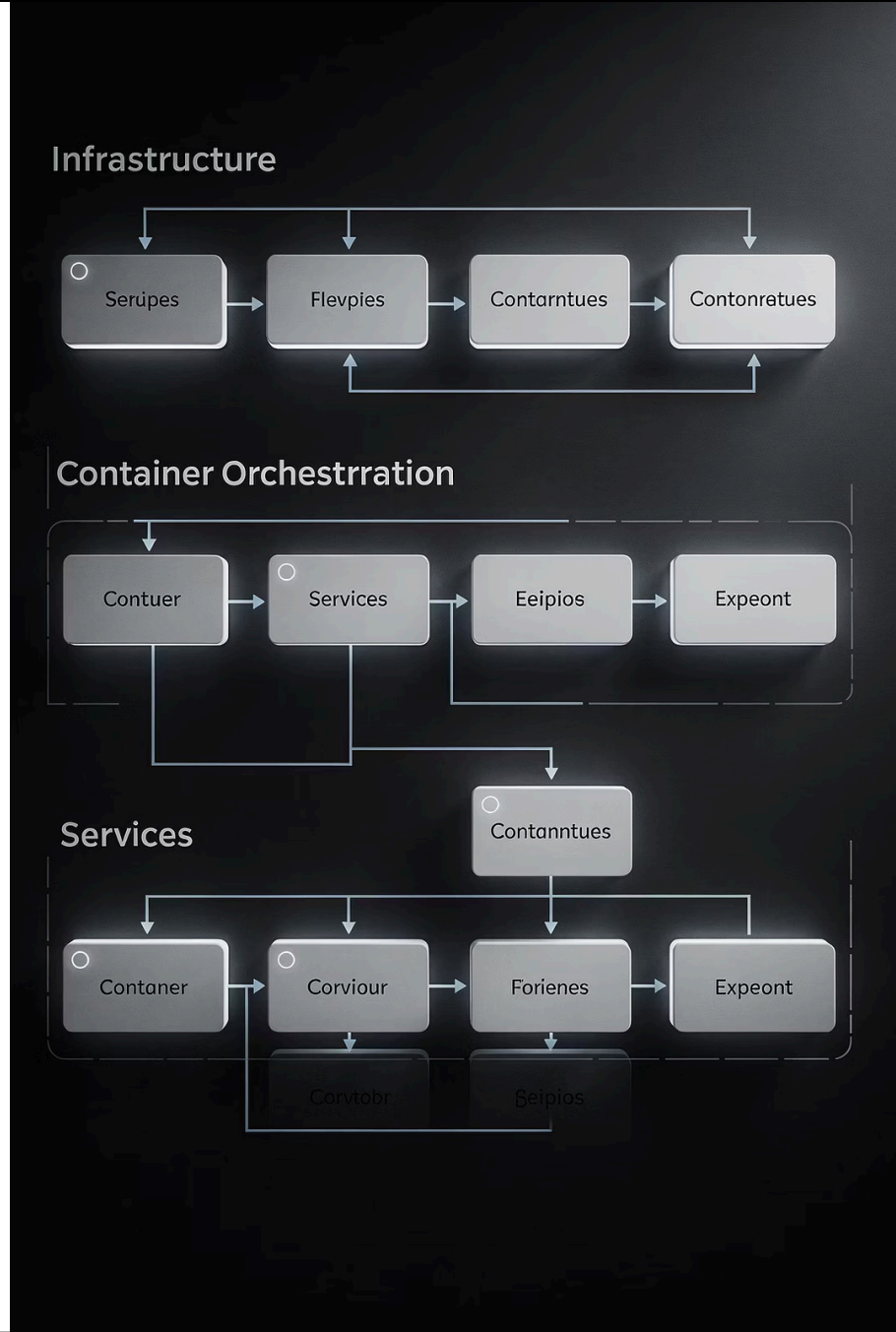
Deploy

Publish to Private Automation Hub & configure in Controller

Execution environments ensure that **automation runs consistently across all environments**.

16.1 Understanding Execution Environments

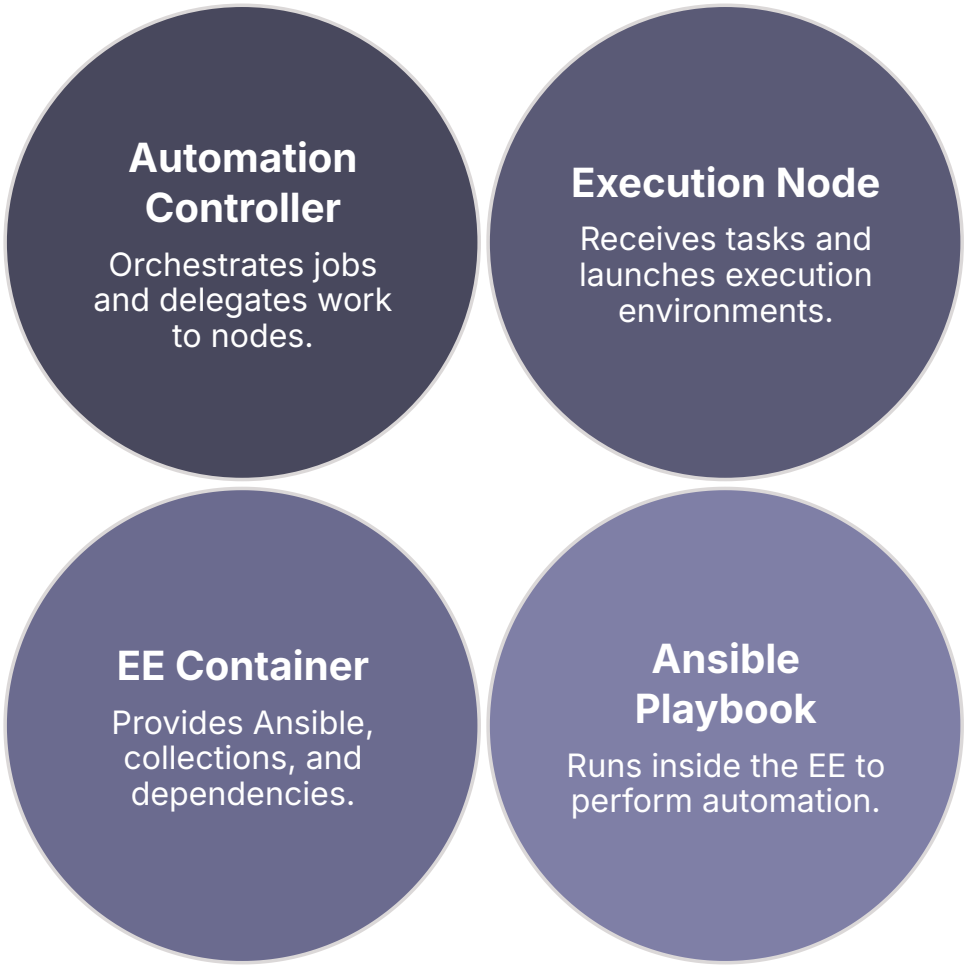
This section covers the foundational concepts behind Execution Environments — what they are, why they exist, and how they differ from legacy approaches.



What Are Execution Environments?

An **Execution Environment (EE)** is a **container image containing everything needed to run Ansible automation** — Ansible core, collections, Python dependencies, and system libraries.

Instead of installing dependencies on the control node, automation runs **inside a container**.



Each automation job runs inside an **isolated container environment**.

Why Use Containers for Automation?

Without Execution Environments

Server1

Python 3.8

Server2

Python 3.9

Server3

Different Ansible version

Automation may fail due to **dependency mismatches**.

With Execution Environments

One Container Image

Python version, Ansible version, Collections, Libraries — all locked in.

Automation runs **exactly the same everywhere**.

Benefits of Containerized Automation

Consistency

Same runtime across all systems

Isolation

Dependencies fully separated

Reproducibility

Same results every time

Portability

Run anywhere containers are supported

Differences from Legacy Virtual Environments

Before Ansible Automation Platform 2.x, automation relied on **Python virtual environments**:

```
virtualenv ansible-env
pip install boto3
pip install netaddr
```

Problems with venvs

- Dependency conflicts between projects
- Difficult environment management
- Different environments per server

Execution environments **replace** this approach entirely.

Virtual Environments vs. Execution Environments

Feature	Virtual Environments	Execution Environments
Isolation	Partial	Full container isolation
Portability	Limited	High
Dependency Control	Hard	Easy
Reproducibility	Difficult	Reliable

Benefits of Execution Environments

Consistent Automation

All playbooks run using the **same environment**, eliminating "works on my machine" issues.

Simplified Dependencies

Dependencies are **packaged once** and reused across all jobs.

Improved Security

Containers **isolate the runtime**, reducing attack surface.

CI/CD Integration

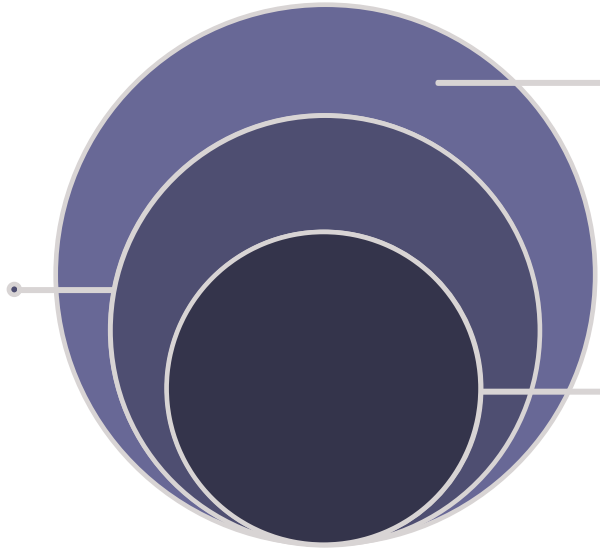
Compatible with **GitLab CI, Jenkins, and GitHub Actions**.

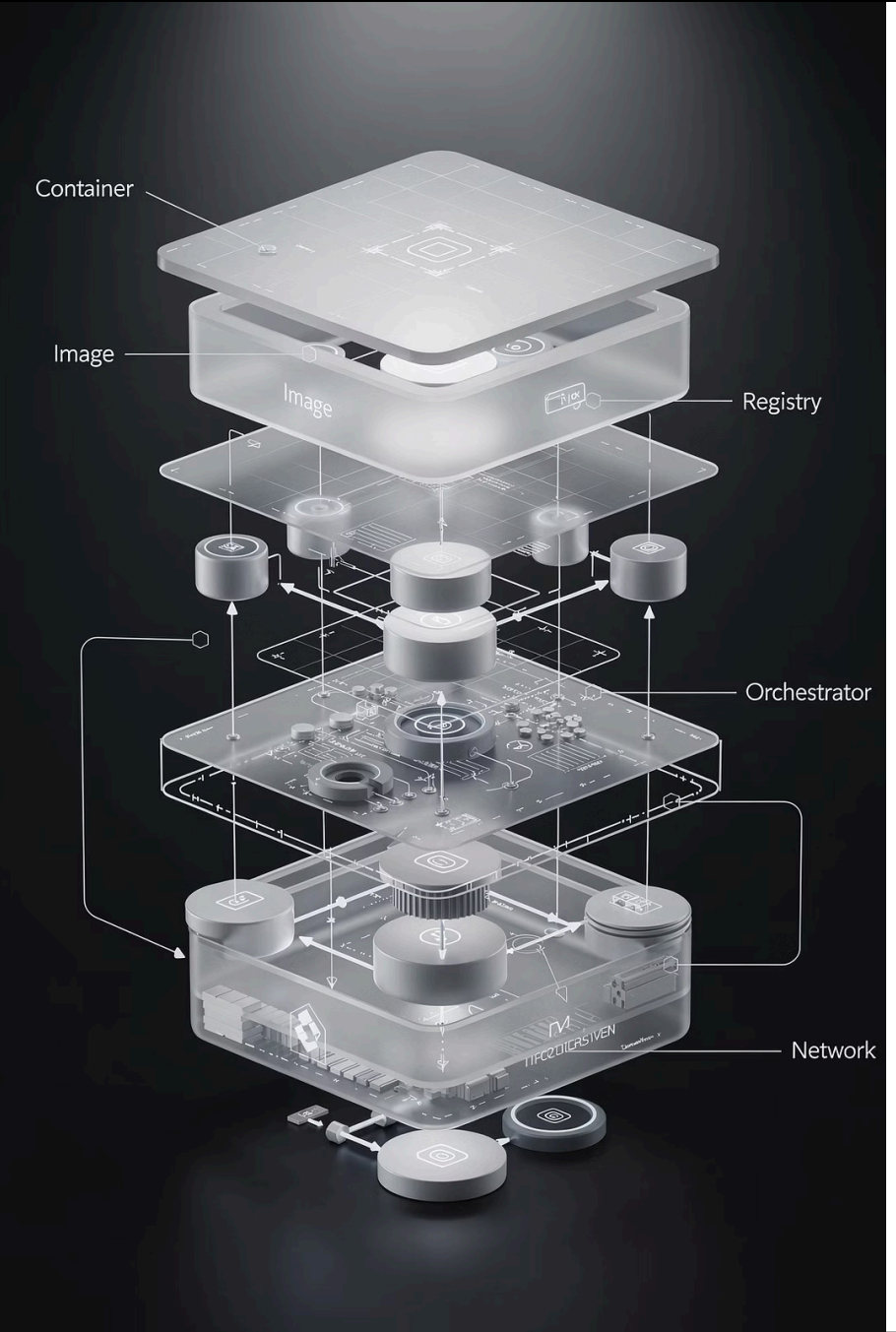
Base Images and Layering Concepts

Execution environments are built from **container base images** such as:

```
quay.io/ansible/ansible-runner
```

Containers use a **layered architecture** — each layer adds functionality, making builds **efficient and reusable**.





16.2 Execution Environment Components

This section explores the internal structure of an execution environment — from Ansible core and collections to Python packages and OS-level dependencies.

Container Image Structure

An execution environment container includes all layers needed to run automation:

```
/usr/bin/ansible
/usr/lib/python
/collections
/site-packages
/system packages
```

Ansible Core

Python

Collections

Python Libraries

OS Packages

Ansible Version and Collections

Each execution environment specifies an **Ansible version**, e.g.:

```
ansible-core 2.15
```

Collections are **packaged inside the EE** and provide the modules used by playbooks.

community.general

amazon.aws

cisco.ios

Python Packages & System Dependencies

Python Libraries

Some modules require Python packages included inside the EE:

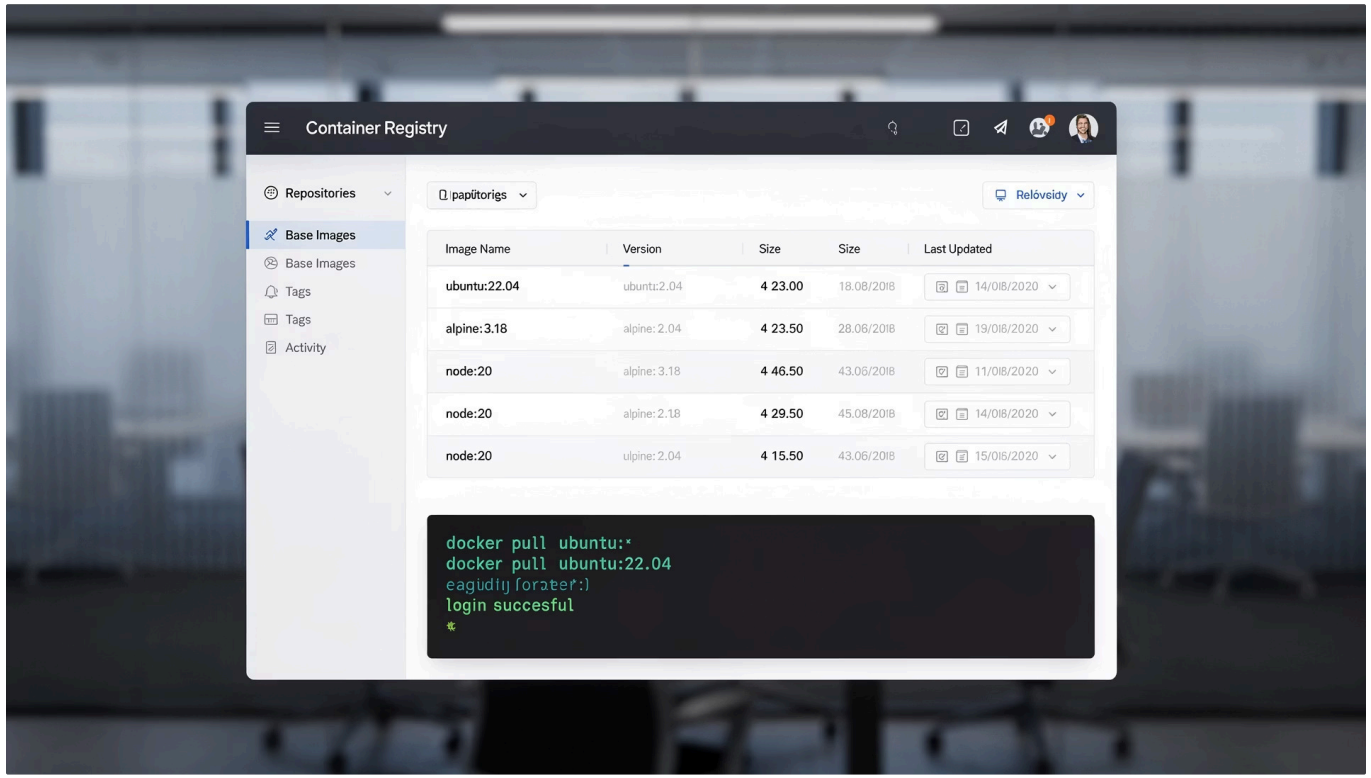
- **boto3**
- **netaddr**
- **requests**
- **pyvmomi**

System Packages (RPM/APT)

Some automation requires OS-level packages installed inside the container:

- **git**
- **sshpass**
- **iproute**
- **net-tools**

Understanding Base Images



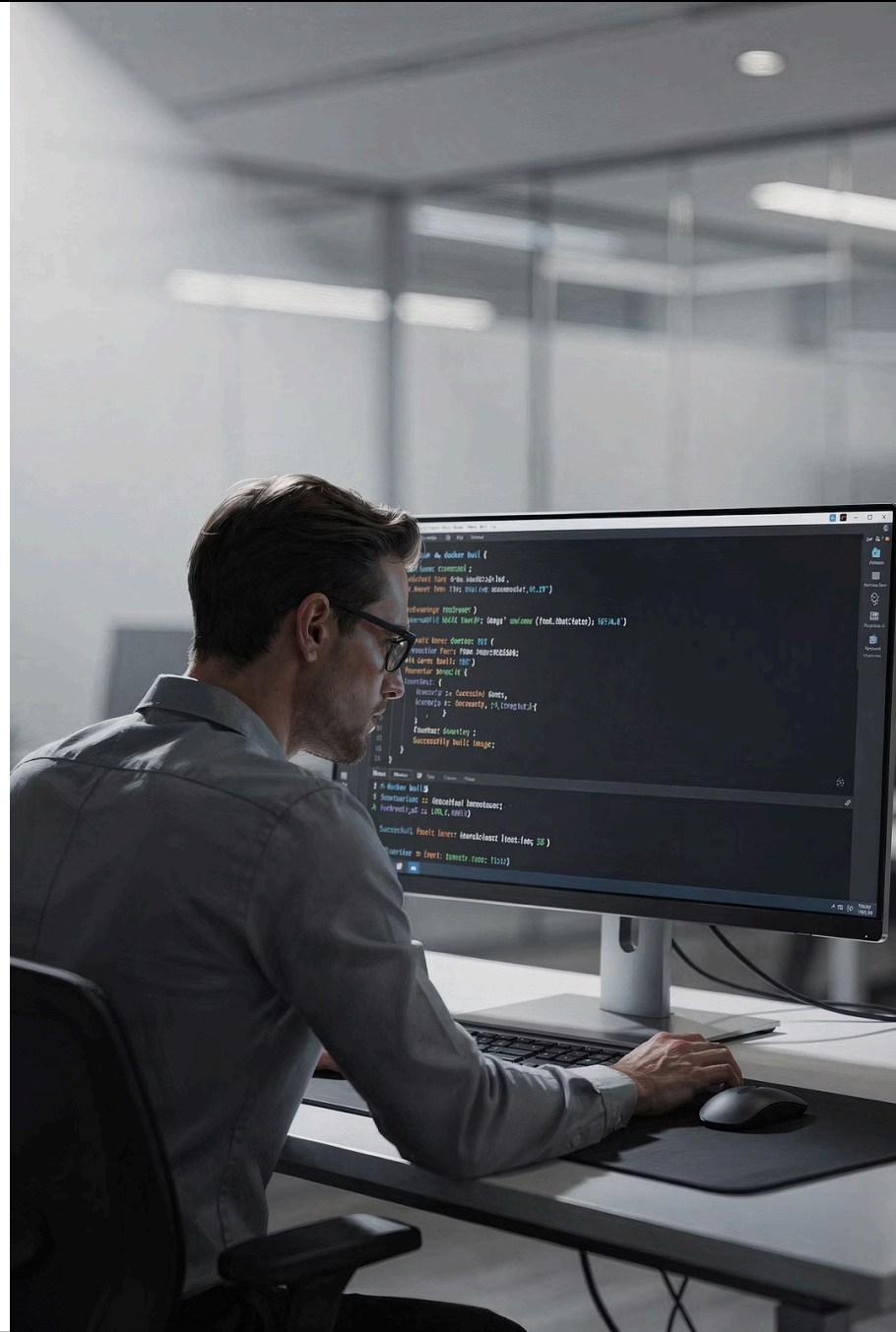
Execution environments start from **official base images**:

```
quay.io/ansible/ansible-runner
quay.io/ansible/ansible-builder
```

These images already include **Python, Ansible core, and basic utilities**. Custom EEs **extend** these images with additional dependencies.

16.3 Building Execution Environments

This section covers how to use **ansible-builder** to create custom execution environments from configuration files.



Introduction to ansible-builder

ansible-builder is the tool used to **create execution environments**. It automates container creation by reading from a configuration file:

```
execution-environment.yml
```

Build command:

```
ansible-builder build
```

→ Define dependencies in **execution-environment.yml**

→ Run **ansible-builder build** to create the image

→ Push the image to **Private Automation Hub**